FATEMEH BAGHERZAD

fa.bagherzad@gmail.com • 647-292-5011 • North York ON
github.com/FatemeBagherzad • linkedin.com/in/fatemeh-bagherzad

FULL-STACK DEVELOPER | REACT | JAVASCRIPT | AI-DIRECTED DEVELOPMENT

PROFILE

- Front-end developer specializing in React, JavaScript and SCSS, building complex interactive interfaces with responsive layouts, full right-to-left language support, and accessibility built in from the component
- Extends past the front end by writing the specification, directing AI coding agents against it, then gating what comes back with automated tests and security scanning; the platform's Django API, Docker environments and AWS deployment were delivered this way
- Builds the systems that make agent-written code dependable: machine-readable engineering standards, custom agent skills, context engineering, MCP tool integration, automated security gating, and verification that ties every claim to the build actually running
- Designs relational database schemas and REST APIs for large-scale data, with attention to indexing, query paths and keyset pagination
- Works in Agile teams through code review, planning and iterative delivery; background in mathematics and technical design, and takes ownership end to end

TECHNICAL SKILLS

Proficient: JavaScript (ES6+), React 18, HTML5, CSS3, SCSS, responsive and RTL layout, accessibility, Node.js, Express, SQL/MySQL, Knex.js, REST APIs, OpenAPI/Swagger, Docker and Docker Compose, Git, GitHub, Vite
Also work with: Python, Django REST Framework, Playwright, Jest, Linux, Nginx, AWS EC2, Terraform, Redis, GitHub Actions
AI-Assisted Development: Claude Code, GitHub Copilot, MCP servers, agent instruction and skills design, parallel agent workflows, context engineering, automated review gating (Aikido)
Tools: Postman, Figma, Photoshop, Jira, Agile/Scrum

PROFESSIONAL EXPERIENCE

Full-Stack Developer — Auto Farms Inc., Toronto | Jan 2019 – Present

- Built the React 18 front end of a collaborative study platform: 210+ components, 40 custom hooks, and an SCSS design-token system driving user-selectable themes and full right-to-left layout in English, Persian and Arabic, with accessibility built into the components from the start (270+ aria-labelled controls, semantic roles, keyboard paths, reduced-motion support)
- Built a windowed reading view over a 6,236-verse Arabic corpus, using reducer-driven virtualization with scroll-position compensation, anchor restoration and batched fetching, so a low-memory device holds a constant DOM node count while scrolling continuously through the full text
- Built an interactive research canvas with pan, zoom, drag, marquee selection and node-to-node connection drawing on pointer events with ResizeObserver and requestAnimationFrame, with full input-modality parity (drag vs. double-tap, right-click vs. 600 ms long-press) and ≥44 px hit targets, fully operable without a mouse
- Extended into backend delivery by directing coding agents: authored the data model and endpoint contracts for a Django REST Framework API of 14 domain modules, 92 models and 170+ OpenAPI-documented endpoints, and set the billion-row design rule agents applied to every query path — composite indexes matched to real query patterns, keyset pagination instead of deep OFFSET, select_related and prefetch_related to eliminate N+1s, database-side aggregation, with Redis reserved for accelerating reads already index-backed
- Specified the authentication and authorization model and validated it against the implementation: per-device sessions with revocation, login lockout, expiring verification and recovery tokens, per-endpoint throttles, and owner, admin, contributor and viewer permissions enforced server-side on every route, plus a stateless signed-ticket anti-bot gate (honeypot, timing check, browser-side proof of work) invisible to the user, since much of the audience is elderly
- Consolidated production, staging and every per-branch environment onto a single parameterised Docker Compose definition of nine services, including Nginx with automated Let's Encrypt renewal, Redis and scheduled workers, with AWS provisioning (EC2, security group, elastic IP, encrypted gp3) expressed as Terraform; shifted integration testing left onto the developer machine, running eight complete environments in parallel on isolated port blocks that carry 115 Playwright spec files and 500+ tests across 6 browser and device profiles pre-push

- Built the operating layer the coding agents run inside: engineering standards encoded as machine-readable rules every change must satisfy, custom agent skills for repeated procedures, subject-routed context loading so a session reads only the design documents its task touches, and a tool-permission policy that withholds commits, pushes and destructive commands from the agent — mirrored across Claude Code and GitHub Copilot with a stated precedence on conflict
- Extended the agent loop with MCP servers, a containerized code-graph index for symbol-level retrieval and Aikido security scanning, and gated every change behind the test suites and that scan before it could land; stamped the built image tag into the served HTML so any claim that a change works is checked against the build actually running
- Ran coding agents in parallel across 9 branch-scoped workspaces, each with its own Docker stack and port block, merging independent features concurrently without shared-state collisions, with a guardrails file of recorded agent failures fed back as permanent rules, a stop condition that reclassifies a twice-failed fix as a product bug, and change-scoped test selection so each agent validates exactly what it touched
- Built a human-in-the-loop LLM pipeline for multilingual content: model drafts are keyed by source sentence in a committed glossary, and once a reviewer corrects an entry it is locked, never re-sent to the model and never overwritten on a later run, so automation scales while human judgement stays authoritative

Software Engineer Teaching Assistant — BrainStation, Toronto | Feb 2024 – Sep 2024

- Mentored students through a full-stack development bootcamp, reviewing code and debugging Node.js, Express, React, Knex and MySQL projects
- Guided capstone projects from design through implementation, building responsive and accessible interfaces
- Coached students on Agile workflow and version control using Jira, Git and GitHub, including branching, pull requests and code review
- Supported students through API testing, CI/CD setup and debugging, coaching them to diagnose problems themselves

Technical Designer — Tehran, Iran | Mar 2006 – May 2015

- Produced 2D and 3D technical visualizations and animations for industrial and architectural projects using AutoCAD, 3ds Max and Adobe tools, and wrote scripts to automate repetitive rendering and file-preparation tasks that other designers adopted
- Progressed from designer to 2D and 3D Design Lead, taking responsibility for project delivery and guiding junior designers

EDUCATION

Software Engineering Diploma — BrainStation, Toronto, 2023 – 94% average

Bachelor's Degree in Pure Mathematics — Iran University of Science and Technology